

Capítulo 1

Marco de Trabajo (Framework)

1.1. Introducción

La instrumentación científica en laboratorios de física experimental enfrenta frecuentemente el desafío de la falta de reproducibilidad y el empaquetamiento propietario...

1.2. Marco Teórico

1.2.1. Paradigma de Co-diseño Hardware/Software

1.2.2. Filosofía y Ecosistema PYNQ

1.2.3. Filosofía y Gestión DevOps con Hog

1.3. Metodología

1.3.1. Estrategia de Repositorios Duales (HW/SW)

1.3.2. Flujo de Trabajo CI/CD y Transducción AXI/DMA

1.3.3. Justificación: Instrumentación Libre, Modular y Reusable

1.4. Resultados y Validación

1.4.1. Plataforma de Adquisición: `hw-xadc-dma-overlays`

Para validar el marco de trabajo propuesto y establecer un entorno preliminar de pruebas para sistemas de adquisición de datos (DAQ), se desarrolló el repositorio de firmware `hw-xadc-dma-overlays`, orientado a la tarjeta SoC-FPGA PYNQ-Z2 (dispositivo Zynq-7000 xc7z020clg400-1). Este proyecto implementa una plataforma de adquisición en la Lógica Programable (PL) capaz de digitalizar señales analógicas en tiempo real y transferirlas hacia la memoria DDR del procesador (PS) sin intervención de la CPU.

Arquitectura de Adquisición y Flujo AXI4-Stream

La arquitectura de adquisición se diseñó utilizando Vivado 2024.2.2 bajo la metodología de IP Integrator (Figura 1.1). El flujo de señal se articula a través de los siguientes módulos principales:

1. **Digitalizador XADC** (`xadc_wiz_0`): Configurado en modo de canal único sobre la entrada auxiliar diferencial `VAUXP1/VAUXN1` (con opción a canal dedicado `VP/VN`), operando a una frecuencia de muestreo de 1 MSPS (1000 kSPS). La interfaz de salida fue establecida en formato AXI4-Stream de 16 bits (`M_AXIS`).
2. **Generador de TLAST** (`tlast_generator`): Debido a que el bloque nativo XADC no genera la señal de final de paquete (`TLAST`), se diseñó

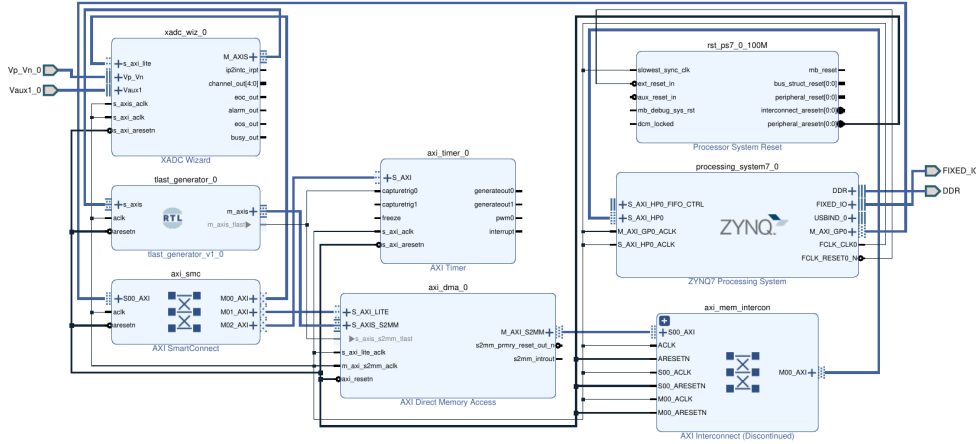


Figura 1.1: Diagrama de bloques general en Vivado IP Integrator para la plataforma de adquisición hw-xadc-dma-overlays.

un módulo VHDL personalizado que actúa como un empaquetador de flujo. Este módulo incrementa un contador síncrono en cada ciclo válido y aserta la señal **TLAST** combinacionalmente cada $N = 16\,384$ muestras (**PACKET_SIZE**), delimitando los bloques de memoria requeridos por el motor DMA.

3. **Acceso Directo a Memoria (axi_dma_0):** El motor AXI DMA opera en modo *Simple DMA* directo (S2MM: *Stream-to-Memory-Mapped*) con un tamaño de ráfaga (*burst size*) de 128 transferencias. Los datos son transmitidos a través del interconector de datos (**axi_mem_intercon**), el cual adapta el flujo de 32 bits del DMA hacia el bus de alto rendimiento AXI-HP0 de 64 bits del Zynq PS. Aunque la CPU ARM Cortex-A9 opera a 32 bits, la transferencia de datos se realiza mediante DMA directamente hacia la memoria RAM compartida (0x00000000 – 0x1FFFFFFF) sobre el bus de 64 bits, liberando por completo a la CPU de tareas de procesamiento de muestras en tiempo real.
4. **Temporizador AXI y Medición de Latencia (axi_timer_0):** Como bloque de apoyo para caracterización, se integró un temporizador AXI cuya entrada de captura (**capturetrig0**) se encuentra interconectada directamente a la señal **TLAST**. Esto permite medir con precisión de ciclos de reloj (10 ns a 100 MHz) la latencia entre paquetes de adquisición desde el hardware.

5. **Infraestructura de Interconexión y Reloj:** El bloque `processing_system7_0` genera un reloj de sistema de 100 MHz (`FCLK_CLK0`) y un reseteo sincronizado mediante el bloque `rst_ps7_0_100M`. La distribución de comandos de control desde el PS se gestiona mediante el conmutador AXI SmartConnect (`axi_smc`) sobre el bus `M_AXI_GP0`, mapeando las direcciones de registro del DMA (`0x40400000`), del temporizador (`0x42800000`) y del XADC (`0x43C00000`).

Gestión de Firmware y Automatización con CERN Hog

La construcción y control de versiones del proyecto de hardware se gestionaron mediante la herramienta CERN Hog. La estructura del repositorio se organiza bajo el directorio `Top/pynq_z2/`, utilizando archivos de listas (`.src`) para desacoplar el código fuente VHDL y el diagrama de bloques (`xadc.bd`) de los archivos binarios propietarios de Vivado.

Adicionalmente, el *wrapper* de nivel superior (`xadc_wrapper.vhd`) incorpora genéricos de versión (`GLOBAL_VER`, `GLOBAL_SHA`), lo que permite embeber dinámicamente el *hash* del commit de Git dentro de los registros del hardware durante la síntesis.

Al finalizar la implementación, un script de gancho (*hook*) en TCL (`post-bitstream.tcl`) extrae el archivo de mapa de bits (`pynq_z2.bit`) y el archivo de descripción de hardware (`pynq_z2.hwh`) hacia la carpeta `bin/`. Finalmente, un flujo de integración continua en GitHub Actions (`release.yml`) empaqueta y publica automáticamente estos artefactos binarios en los lanzamientos del repositorio al detectar una etiqueta de versión (`v*`).

1.4.2. Interfaz e Instrumentación: sw-pynq-oscilloscope

1.4.3. Proyección a Módulos de Localización Espacial

1.5. Conclusiones del Capítulo